

Punto 2.10 - Filtros globales de consulta

Ejercicio guiado: introducir filtros globales de consulta en el modelo acumulativo de AceriaData y comprobar que EF Core oculta determinadas filas en las consultas normales sin eliminarlas físicamente de SQL Server.

Contexto del proyecto: 2.10 parte íntegramente del estado 2.9. Se conservan claves, relaciones, índices y restricciones CHECK. En este punto no se añaden propiedades de Soft Delete: se reutilizan propiedades de negocio ya existentes para definir cuatro filtros reales con `HasQueryFilter`. El siguiente punto, 2.11, será el que introduzca `IsDeleted` y `DeletedAt`.

Objetivos de aprendizaje

- Comprender qué hace un filtro global de consulta en EF Core.
- Configurar `HasQueryFilter` sobre varias entidades.
- Filtrar órdenes cuyo `Estado` sea `Cancelada`.
- Mostrar sólo `PlanchaAcero` con `Activa == true`.
- Mostrar sólo `EstadoOrden` con `Activo == true`.
- Mostrar sólo relaciones `OrdenAleacion` cuyo `EstadoRelacion` sea `Activa`.
- Usar `IgnoreQueryFilters()` de forma explícita.
- Diferenciar ocultación lógica de eliminación física.
- Comprobar la diferencia entre una consulta normal y la misma consulta sin filtros.
- Entender por qué los filtros globales no requieren modificar el esquema físico.
- Interpretar correctamente la migración histórica vacía `M2_2_10`.
- Mantener separado este punto del Soft Delete que comienza en 2.11.

Paso 1: Abrir el estado autónomo de 2.10

```
cd M02/PROYECTO/2.10
dotnet restore AceriaData.sln
dotnet build AceriaData.sln --configuration Release
```

El checkpoint contiene el estado completo del punto. Si reproduces manualmente la evolución, parte de 2.9 y añade únicamente los filtros descritos aquí.

Paso 2: Delimitar el cambio respecto a 2.9

En 2.9 el esquema quedó consolidado con claves, índices y restricciones CHECK. En 2.10 **no se rehace ninguna de esas configuraciones**.

El cambio nuevo es de comportamiento de consulta:

- `OrdenFabricacion`: ocultar estados `Cancelada`;
- `PlanchaAcero`: ocultar planchas inactivas;
- `EstadoOrden`: ocultar estados cuyo `Activo` sea falso;
- `OrdenAleacion`: ocultar relaciones cuyo `EstadoRelacion` no sea `Activa`.

Todavía no existen `IsDeleted` ni `DeletedAt`. Añadirlas aquí adelantaría el contenido de 2.11 y haría divergir la práctica del checkpoint real.

Paso 3: Configurar el filtro de OrdenFabricacion

El modelo efectivo contiene:

```
modelBuilder.Entity<OrdenFabricacion>(entity =>
{
    // ... configuración heredada ...
    entity.HasQueryFilter(
        o => o.Estado != "Cancelada");
});
```

La condición se incorpora automáticamente a las consultas LINQ normales de `OrdenFabricacion`. La fila cancelada sigue almacenada en la tabla; simplemente queda fuera de la vista habitual de EF Core.

Paso 4: Configurar el filtro de PlanchaAcero

```
modelBuilder.Entity<PlanchaAcero>(entity =>
{
    // ... configuración heredada ...
    entity.HasQueryFilter(x => x.Activa);
});
```

La propiedad `Activa` ya formaba parte del modelo antes de este punto. El filtro reutiliza esa propiedad: no añade una columna nueva.

Una consulta normal sobre `PlanchasAcero` devuelve sólo filas activas. Para una operación administrativa que necesite ver también las inactivas se puede usar `IgnoreQueryFilters()`.

Paso 5: Configurar el filtro de EstadoOrden

El checkpoint real contiene también:

```
modelBuilder.Entity<EstadoOrden>(entity =>
{
    // ... configuración heredada ...
    entity.HasQueryFilter(e => e.Activo);
});
```

Por tanto, 2.10 no trabaja únicamente con órdenes y planchas. Los estados marcados como no activos también quedan ocultos en consultas normales de `EstadoOrden`.

Paso 6: Configurar el filtro de OrdenAleacion

```
modelBuilder.Entity<OrdenAleacion>(entity =>
{
    // ... configuración heredada ...
    entity.HasQueryFilter(
        x => x.EstadoRelacion == "Activa");
});
```

La relación muchos-a-muchos con payload conserva su PK compuesta, sus FK y sus índices de 2.9. El único cambio de este punto es la regla de visibilidad basada en `EstadoRelacion`.

Paso 7: Comprender qué entidades no reciben un filtro nuevo

En el estado 2.10 no se configura un `HasQueryFilter` nuevo para:

- `Aleacion`;
- `DetalleOrden`;
- `CertificadoCalidad`.

Esto es deliberado. La práctica debe representar el modelo real y no añadir reglas que no aparecen en `M02/PROYECTO0/2.10/Program.cs`.

| Paso 8: Comparar una consulta normal con IgnoreQueryFilters

El ejemplo ejecutable usa:

```
var visibles =
    context.OrdenesFabricacion.Count();

var todas =
    context.OrdenesFabricacion
        .IgnoreQueryFilters()
        .Count();
```

La primera consulta aplica `Estado != "Cancelada"`. La segunda desactiva expresamente los filtros globales para esa consulta.

`IgnoreQueryFilters()` debe utilizarse con intención: sirve para diagnósticos, administración o escenarios que necesitan acceder a filas normalmente ocultas. No conviene convertirlo en la forma habitual de consultar porque neutraliza la regla transversal.

| Paso 9: Preparar una fila visible y otra cancelada

El checkpoint crea primero la orden normal de validación y después una segunda orden:

```
var cancelada = new OrdenFabricacion
{
    NumeroOrden = "OF-M2-CANCELADA",
    Cliente = "Cliente Histórico",
    FechaCreacion = DateTime.UtcNow,
    Estado = "Cancelada"
};

context.OrdenesFabricacion.Add(cancelada);
context.SaveChanges();
```

La operación `SaveChanges()` persiste la fila. El filtro no impide insertar una orden cancelada: actúa cuando EF Core compone consultas.

| Paso 10: Compilar antes de revisar el historial

```
dotnet build AceriaData.sln --configuration Release
```

La compilación debe terminar correctamente antes de ejecutar las herramientas de EF Core.

| Paso 11: Entender la migración histórica M2_2_10

El checkpoint contiene:

```
Migrations/20260927204833_M2_2_10.cs
```

pero su contenido real es:

```
protected override void Up(
    MigrationBuilder migrationBuilder)
{
}

protected override void Down(
    MigrationBuilder migrationBuilder)
{
}
```

Esto es coherente con el cambio introducido: `HasQueryFilter` modifica el modelo de consulta de EF Core, pero **no crea columnas, índices, claves ni restricciones en SQL Server**.

Si reproduces literalmente el historial del curso desde 2.9, `dotnet ef migrations add M2_2_10` genera una migración sin operaciones de esquema. El checkpoint la conserva como hito histórico; no debe confundirse con una migración que altere físicamente la base.

Paso 12: Aplicar y revisar el historial acumulativo

```
dotnet ef database update
dotnet ef migrations list
```

La lista incluye `M2_2_10` después de `M2_2_9`, aunque `Up()` esté vacío. El esquema físico sigue siendo el consolidado en 2.9.

Durante la ejecución del proyecto se mantiene `Database.Migrate()`. No se sustituye el historial por `EnsureCreated()`.

Paso 13: Ejecutar el escenario completo

```
dotnet run --project AceriaData.Console.csproj --configuration Release
```

El escenario crea una orden visible y otra cancelada, ejecuta ambos recuentos y termina con:

```
2.10 OK | Visibles: 1 | Sin filtro: 2
```

La diferencia demuestra que la fila cancelada existe pero el filtro global la excluye de la consulta normal.

Paso 14: Inspeccionar el SQL generado

El proyecto mantiene logging de comandos de EF Core. Al ejecutar el escenario, compara el SQL del recuento normal con el recuento que usa `IgnoreQueryFilters()`.

La consulta normal incorpora el predicado derivado de:

```
o => o.Estado != "Cancelada"
```

La consulta con `IgnoreQueryFilters()` no aplica ese filtro global. Esta evidencia permite comprobar el comportamiento real sin confundirlo con borrado físico.

Paso 15: Probar los otros filtros sin cambiar el modelo final

La misma idea puede comprobarse sobre las otras entidades configuradas:

```
var planchasVisibles =
    context.PlanchasAcero.Count();

var planchasTotales =
    context.PlanchasAcero
        .IgnoreQueryFilters()
        .Count();

var estadosVisibles =
    context.EstadosOrden.Count();

var estadosTotales =
    context.EstadosOrden
        .IgnoreQueryFilters()
        .Count();
```

No es necesario añadir nuevas propiedades ni otra migración. El objetivo es observar cómo el mismo modelo cambia la visibilidad según se apliquen o no los filtros.

Paso 16: Resolver el reto de la orden cancelada

El checkpoint contiene el bloque comentado `RETO 2.10 - ORDEN CANCELADA E IGNOREQUERYFILTERS`.

Al activarlo se inserta otra orden cancelada:

```
var canceladaReto = new OrdenFabricacion
{
    NumeroOrden = "OF-M2-CANCELADA-RETO",
    Cliente = "Cliente Histórico Reto",
    FechaCreacion = DateTime.UtcNow,
    Estado = "Cancelada"
};

context.OrdenesFabricacion.Add(canceladaReto);
context.SaveChanges();

var visiblesReto =
    context.OrdenesFabricacion.Count();

var todasReto =
    context.OrdenesFabricacion
        .IgnoreQueryFilters()
        .Count();
```

La consulta normal mantiene el mismo número de órdenes visibles; la consulta sin filtros aumenta porque incorpora también la nueva fila cancelada.

Paso 17: Delimitar el alcance antes de 2.11

Al terminar 2.10:

- no se añaden `IsDeleted` ni `DeletedAt`;
- no se modifica el esquema consolidado en 2.9;
- no se sustituyen los filtros de negocio por Soft Delete;
- no se elimina físicamente una orden por estar cancelada;
- no se usa `IgnoreQueryFilters()` de forma indiscriminada;
- no se usa `EnsureCreated()` para evitar Migrations.

2.11 partirá de este estado y añadirá el Soft Delete real del curso.

Errores comunes

Error	Causa	Corrección
Pensar que una fila filtrada fue eliminada	La consulta normal no la devuelve	Comprobarla con <code>IgnoreQueryFilters()</code>
Esperar cambios de tablas al añadir <code>HasQueryFilter</code>	Un filtro global modifica consultas, no el esquema físico	Revisar que <code>M2_2_10.Up()</code> y <code>Down()</code> están vacíos
Añadir <code>IsDeleted</code> en 2.10	Se adelanta el contenido de 2.11	Mantener sólo propiedades de negocio ya existentes
Olvidar los filtros de <code>EstadoOrden</code> u <code>OrdenAleacion</code>	Se documenta sólo una parte del modelo real	Contrastar el MD con <code>Program.cs</code> del checkpoint
Usar <code>IgnoreQueryFilters()</code> en todas las consultas	Se neutralizan las reglas transversales	Reservarlo para escenarios que realmente necesiten filas ocultas

Error	Causa	Corrección
Usar <code>EnsureCreated()</code>	Se rompe la evolución acumulativa	Mantener <code>Migrations</code> y <code>Database.Migrate()</code>

Analugía final

Un filtro global se parece a una vista de trabajo automática: las filas siguen en el almacén, pero EF Core aplica una regla de visibilidad cada vez que consulta esa entidad. `IgnoreQueryFilters()` equivale a abrir temporalmente la vista completa.

Resultado esperado

Al finalizar 2.10, `AceriaData` conserva íntegro el esquema de 2.9 y añade cuatro reglas de visibilidad en el modelo de EF Core. La ejecución demuestra que una orden cancelada permanece en SQL Server pero queda fuera de la consulta normal. El marcador esperado es:

2.10 OK | Visibles: 1 | Sin filtro: 2

Conexión con el siguiente punto

2.11 parte de este estado para introducir Soft Delete con `IsDeleted`, `DeletedAt` y los cambios de esquema correspondientes, sin perder los conceptos de filtros globales aprendidos aquí.

Código acumulativo completo del estado 2.10

El siguiente archivo es el `Program.cs` real del estado ejecutable del punto. Se incluye para que la práctica y el proyecto ejecutable no diverjan.

```
using System.ComponentModel.DataAnnotations;
using System.ComponentModel.DataAnnotations.Schema;
using Microsoft.EntityFrameworkCore;
using Microsoft.Extensions.Configuration;
using Microsoft.Extensions.DependencyInjection;
using Microsoft.Extensions.Logging;

namespace AceriaData.ConsoleApp;

[Table("OrdenesFabricacion")]
public class OrdenFabricacion
{
    [Key]
    public int Id { get; set; }
    [Required]
    [MaxLength(50)]
    public string NumeroOrden { get; set; } = string.Empty;
    [Required]
    [MaxLength(200)]
    public string Cliente { get; set; } = string.Empty;
    public DateTime FechaCreacion { get; set; }
    public DateTime? FechaEntrega { get; set; }
    [Required]
    [MaxLength(50)]
    public string Estado { get; set; } = "Pendiente";
    [MaxLength(500)]
    public string? Observaciones { get; set; }
    public List<PlanchaAcero> Planchas { get; set; } = new();
    public DetalleOrden? Detalle { get; set; }
    public CertificadoCalidad? Certificado { get; set; }
    public List<OrdenAleacion> OrdenesAleaciones { get; set; } = new();
}

[Table("PlanchasAcero")]
public class PlanchaAcero
{

```

```

    [Key]
    public int Id { get; set; }
    public int OrdenId { get; set; }
    public double Espesor { get; set; }
    public double Ancho { get; set; }
    public double Largo { get; set; }
    [Precision(18, 3)]
    public decimal Peso { get; set; }
    public bool Activa { get; set; } = true;
    [ForeignKey(nameof(OrdenId))]
    public OrdenFabricacion Orden { get; set; } = null!;
}

[Table("Aleaciones")]
public class Aleacion
{
    [Key]
    public int Id { get; set; }
    [Required]
    [MaxLength(100)]
    public string Nombre { get; set; } = string.Empty;
    [Required]
    [MaxLength(20)]
    public string Codigo { get; set; } = string.Empty;
    public double PorcentajeCarbono { get; set; }
    public double PorcentajeManganeso { get; set; }
    [MaxLength(500)]
    public string? Descripcion { get; set; }
    public List<OrdenAleacion> OrdenesAleaciones { get; set; } = new();
}

[Table("EstadosOrden")]
public class EstadoOrden
{
    [Key]
    public int Id { get; set; }
    [Required]
    [MaxLength(50)]
    public string Nombre { get; set; } = string.Empty;
    [MaxLength(250)]
    public string Descripcion { get; set; } = string.Empty;
    public bool Activo { get; set; } = true;
}

[Table("DetallesOrden")]
public class DetalleOrden
{
    [Key]
    public int Id { get; set; }
    public int OrdenId { get; set; }
    [Required]
    [MaxLength(200)]
    public string ComposicionQuimica { get; set; } = string.Empty;
    public double TemperaturaColada { get; set; }
    [MaxLength(500)]
    public string? Notas { get; set; }
    [ForeignKey(nameof(OrdenId))]
    public OrdenFabricacion Orden { get; set; } = null!;
}

[Table("CertificadosCalidad")]
public class CertificadoCalidad
{
    [Key]
    public int Id { get; set; }
    public int OrdenId { get; set; }
    [Required]
    [MaxLength(50)]
    public string NumeroCertificado { get; set; } = string.Empty;
    public DateTime FechaEmision { get; set; }
    [Required]
    [MaxLength(100)]
    public string OrganismoCertificador { get; set; } = string.Empty;
}

```

```

    [ForeignKey(nameof(OrdenId))]
    public OrdenFabricacion Orden { get; set; } = null!;
}

[PrimaryKey(nameof(OrdenFabricacionId), nameof(AleacionId))]
[Table("OrdenesAleaciones")]
public class OrdenAleacion
{
    public int OrdenFabricacionId { get; set; }
    public int AleacionId { get; set; }
    public DateTime FechaAsignacion { get; set; } = DateTime.Now;
    [Precision(18, 3)]
    public decimal CantidadUtilizada { get; set; }
    [MaxLength(20)]
    public string EstadoRelacion { get; set; } = "Activa";
    public OrdenFabricacion Orden { get; set; } = null!;
    public Aleacion Aleacion { get; set; } = null!;
}

public class AceriaDbContext : DbContext
{
    public AceriaDbContext(DbContextOptions<AceriaDbContext> options) : base(options) { }

    public DbSet<OrdenFabricacion> OrdenesFabricacion => Set<OrdenFabricacion>();
    public DbSet<PlanchaAcero> PlanchasAcero => Set<PlanchaAcero>();
    public DbSet<Aleacion> Aleaciones => Set<Aleacion>();
    public DbSet<EstadoOrden> EstadosOrden => Set<EstadoOrden>();
    public DbSet<DetalleOrden> DetallesOrden => Set<DetalleOrden>();
    public DbSet<CertificadoCalidad> CertificadosCalidad => Set<CertificadoCalidad>();
    public DbSet<OrdenAleacion> OrdenesAleaciones => Set<OrdenAleacion>();

    protected override void OnModelCreating(ModelBuilder modelBuilder)
    {
        base.OnModelCreating(modelBuilder);
        modelBuilder.Entity<OrdenFabricacion>(entity =>
        {
            entity.ToTable("OrdenesFabricacion");
            entity.HasKey(o => o.Id);
            entity.Property(o => o.NumeroOrden).IsRequired().HasMaxLength(50);
            entity.Property(o => o.Cliente).IsRequired().HasMaxLength(200);
            entity.Property(o => o.FechaCreacion).HasDefaultValueSql("GETDATE()");
            entity.Property(o => o.Estado).IsRequired().HasMaxLength(50).HasDefaultValue("Pendiente");
            entity.Property(o => o.Observaciones).HasMaxLength(500);
            entity.HasAlternateKey(o => o.NumeroOrden).HasName("AK_OrdenesFabricacion_NumeroOrden");
            entity.HasIndex(o => o.Cliente).HasDatabaseName("IX_OrdenesFabricacion_Cliente");
            entity.HasIndex(o => new { o.Cliente,
o.FechaCreacion }).HasDatabaseName("IX_OrdenesFabricacion_Cliente_FechaCreacion");
            entity.HasIndex(o => o.FechaEntrega).HasFilter("[Estado] = 'Pendiente'").HasDatabaseName("IX_OrdenesFabricacion_FechaEntrega_Pendientes");
            entity.HasIndex(o => o.Estado).IncludeProperties(o => new { o.NumeroOrden, o.Cliente,
o.FechaCreacion }).HasDatabaseName("IX_OrdenesFabricacion_Estado_Incluye");
            entity.HasQueryFilter(o => o.Estado != "Cancelada");
        });
        modelBuilder.Entity<PlanchaAcero>(entity =>
        {
            entity.ToTable("PlanchasAcero", t =>
            {
                t.HasCheckConstraint("CK_PlanchasAcero_Espesor", "[Espesor] > 0");
                t.HasCheckConstraint("CK_PlanchasAcero_Ancho", "[Ancho] > 0");
                t.HasCheckConstraint("CK_PlanchasAcero_Largo", "[Largo] > 0");
                t.HasCheckConstraint("CK_PlanchasAcero_Peso", "[Peso] > 0");
            });
            entity.HasKey(x => x.Id);
            entity.Property(x => x.Peso).HasPrecision(18, 3);
            entity.Property(x => x.Activa).HasDefaultValue(true);
            entity.HasOne(x => x.Orden)
                .WithMany(o => o.Planchas)
                .HasForeignKey(x => x.OrdenId)
                .OnDelete(DeleteBehavior.Cascade)
                .IsRequired();
            entity.HasIndex(x => new { x.OrdenId,
x.Activa }).HasDatabaseName("IX_PlanchasAcero_OrdenId_Activa");
        });
    }
}

```

```

        entity.HasIndex(x => x.Espesor).HasFilter("[Activa] = 1").HasDatabaseName("IX_PlanchasAcero_Espesor_Activas");
        entity.HasQueryFilter(x => x.Activa);
    });
    modelBuilder.Entity<Aleacion>(entity =>
    {
        entity.ToTable("Aleaciones", t =>
        {
            t.HasCheckConstraint("CK_Aleaciones_PorcentajeCarbono", "[PorcentajeCarbono] >= 0 AND [PorcentajeCarbono] <= 2");
            t.HasCheckConstraint("CK_Aleaciones_PorcentajeManganeso", "[PorcentajeManganeso] >= 0 AND [PorcentajeManganeso] <= 5");
        });
        entity.HasKey(a => a.Id);
        entity.Property(a => a.Nombre).IsRequired().HasMaxLength(100);
        entity.Property(a => a.Codigo).IsRequired().HasMaxLength(20);
        entity.Property(a => a.Descripcion).HasMaxLength(500);
        entity.HasAlternateKey(a => a.Codigo).HasName("AK_Aleaciones_Codigo");
        entity.HasIndex(a => a.Nombre).HasDatabaseName("IX_Aleaciones_Nombre");
        entity.HasIndex(a => new { a.PorcentajeCarbono, a.PorcentajeManganeso }).HasDatabaseName("IX_Aleaciones_Porcentajes");
    });
    modelBuilder.Entity<EstadoOrden>(entity =>
    {
        entity.ToTable("EstadosOrden");
        entity.HasKey(e => e.Id);
        entity.Property(e => e.Nombre).IsRequired().HasMaxLength(50);
        entity.Property(e => e.Descripcion).HasMaxLength(250);
        entity.Property(e => e.Activo).HasDefaultValue(true);
        entity.HasQueryFilter(e => e.Activo);
    });
    modelBuilder.Entity<DetalleOrden>(entity =>
    {
        entity.ToTable("DetallesOrden");
        entity.HasKey(d => d.Id);
        entity.Property(d => d.ComposicionQuimica).IsRequired().HasMaxLength(200);
        entity.Property(d => d.Notas).HasMaxLength(500);
        entity.HasOne(d => d.Orden)
            .WithOne(o => o.Detalle)
            .HasForeignKey<DetalleOrden>(d => d.OrdenId)
            .OnDelete(DeleteBehavior.Cascade)
            .IsRequired();
    });
    modelBuilder.Entity<CertificadoCalidad>(entity =>
    {
        entity.ToTable("CertificadosCalidad");
        entity.HasKey(c => c.Id);
        entity.Property(c => c.NumeroCertificado).IsRequired().HasMaxLength(50);
        entity.Property(c => c.OrganismoCertificador).IsRequired().HasMaxLength(100);
        entity.HasAlternateKey(c => c.NumeroCertificado).HasName("AK_CertificadosCalidad_NumeroCertificado");
        entity.HasIndex(c => c.FechaEmision).HasDatabaseName("IX_CertificadosCalidad_FechaEmision");
        entity.HasOne(c => c.Orden)
            .WithOne(o => o.Certificado)
            .HasForeignKey<CertificadoCalidad>(c => c.OrdenId)
            .OnDelete(DeleteBehavior.Cascade)
            .IsRequired();
    });
    modelBuilder.Entity<OrdenAleacion>(entity =>
    {
        entity.ToTable("OrdenesAleaciones");
        entity.HasKey(x => new { x.OrdenFabricacionId, x.AleacionId });
        entity.Property(x => x.FechaAsignacion).HasDefaultValueSql("GETDATE()");
        entity.Property(x => x.CantidadUtilizada).HasPrecision(18, 3);
        entity.Property(x => x.EstadoRelacion).IsRequired().HasMaxLength(20).HasDefaultValue("Activa");
        entity.HasOne(x => x.Orden)
            .WithMany(o => o.OrdenesAleaciones)
            .HasForeignKey(x => x.OrdenFabricacionId)
            .OnDelete(DeleteBehavior.Cascade);
        entity.HasOne(x => x.Aleacion)
            .WithMany(a => a.OrdenesAleaciones)

```

```

        .HasForeignKey(x => x.AleacionId)
        .OnDelete(DeleteBehavior.Restrict);
entity.HasIndex(x => x.AleacionId).HasDatabaseName("IX_OrdenesAleaciones_AleacionId");
entity.HasIndex(x => x.EstadoRelacion).HasFilter("[EstadoRelacion] = 'Activa'").HasDatabaseName("IX_OrdenesAleaciones_EstadoRelacion_Activas");
entity.HasQueryFilter(x => x.EstadoRelacion == "Activa");
    });
}
}

public interface IOordenRepositorio
{
    List<OrdenFabricacion> ObtenerTodas();
    OrdenFabricacion? ObtenerPorId(int id);
    OrdenFabricacion? ObtenerPorNumero(string numeroOrden);
    void Agregar(OrdenFabricacion orden);
    void Eliminar(OrdenFabricacion orden);
    int Guardar();
}

public sealed class OrdenRepositorio : IOordenRepositorio
{
    private readonly AceriaDbContext _context;
    public OrdenRepositorio(AceriaDbContext context) => _context = context;

    public List<OrdenFabricacion> ObtenerTodas() => _context.OrdenesFabricacion.OrderBy(o => o.Id).ToList();
    public OrdenFabricacion? ObtenerPorId(int id) => _context.OrdenesFabricacion.Find(id);
    public OrdenFabricacion? ObtenerPorNumero(string numeroOrden) =>
        _context.OrdenesFabricacion.FirstOrDefault(o => o.NumeroOrden == numeroOrden);
    public void Agregar(OrdenFabricacion orden) => _context.OrdenesFabricacion.Add(orden);
    public void Eliminar(OrdenFabricacion orden) => _context.OrdenesFabricacion.Remove(orden);
    public int Guardar() => _context.SaveChanges();
}

public interface IServicioOrdenes
{
    string ObtenerResumen(int ordenId);
}

public sealed class ServicioOrdenes : IServicioOrdenes
{
    private readonly IOordenRepositorio _repositorio;
    private readonly AceriaDbContext _context;
    public ServicioOrdenes(IOordenRepositorio repositorio, AceriaDbContext context)
    {
        _repositorio = repositorio;
        _context = context;
    }

    public string ObtenerResumen(int ordenId)
    {
        var orden = _repositorio.ObtenerPorId(ordenId);
        if (orden is null) return $"Orden {ordenId} no encontrada";
        var totalPlanchas = _context.PlanchasAcero.Count(x => x.OrdenId == ordenId);
        return $"Orden {orden.NumeroOrden} | Cliente: {orden.Cliente} | Planchas: {totalPlanchas}";
    }
}

public static class Program
{
    public static void Main()
    {
        var configuration = new ConfigurationBuilder()
            .SetBasePath(AppContext.BaseDirectory)
            .AddJsonFile("appsettings.json", optional: false, reloadOnChange: false)
            .AddEnvironmentVariables()
            .Build();

        var connectionString = configuration.GetConnectionString("AceriaDB")
            ?? throw new InvalidOperationException("No se encontró la cadena de conexión 'AceriaDB'.");

        var services = new ServiceCollection();

```

```

services.AddDbContext<AceriaDbContext>(options =>
    options
        .UseSqlServer(connectionString, sql =>
            {
                sql.EnableRetryOnFailure(5, TimeSpan.FromSeconds(10), null);
                sql.CommandTimeout(60);
            })
        .LogTo(global::System.Console.WriteLine, new[] {
"Microsoft.EntityFrameworkCore.Database.Command" }, LogLevel.Information)
        .EnableDetailedErrors());

services.AddScoped<IOrdenRepositorio, OrdenRepositorio>();
services.AddScoped<IServicioOrdenes, ServicioOrdenes>();

using var provider = services.BuildServiceProvider(new ServiceProviderOptions
{
    ValidateScopes = true,
    ValidateOnBuild = true
});

using var scope = provider.CreateScope();
var context = scope.ServiceProvider.GetRequiredService<AceriaDbContext>();
context.Database.EnsureDeleted();
context.Database.Migrate();

var orden = new OrdenFabricacion
{
    NumeroOrden = "OF-M2-0001",
    Cliente = "Constructora del Norte",
    FechaCreacion = DateTime.UtcNow,
    FechaEntrega = DateTime.Today.AddDays(14),
    Estado = "Pendiente",
    Observaciones = "Orden de validación M2",
    Planchas =
    {
        new PlanchaAcero
        {
            Espesor = 10.5,
            Ancho = 1500,
            Largo = 3000,
            Peso = 371.250m,
            Activa = true,
        }
    },
    Detalle = new DetalleOrden
    {
        ComposicionQuimica = "C: 0.45%, Mn: 0.75%",
        TemperaturaColada = 1550.5,
        Notas = "Colada principal"
    },
    Certificado = new CertificadoCalidad
    {
        NumeroCertificado = "CERT-0001",
        FechaEmision = DateTime.Today,
        OrganismoCertificador = "Laboratorio Aceria"
    },
};

var aleacion = new Aleacion
{
    Nombre = "AISI 1045",
   Codigo = "A1045",
    PorcentajeCarbono = 0.45,
    PorcentajeManganeso = 0.75,
    Descripcion = "Acero medio en carbono"
};
orden.OrdenesAleaciones.Add(new OrdenAleacion
{
    Aleacion = aleacion,
    CantidadUtilizada = 1500.500m,
    EstadoRelacion = "Activa"
});
context.OrdenesFabricacion.Add(orden);

```

```

        context.EstadosOrden.Add(new EstadoOrden { Nombre = "Pendiente", Descripcion = "Orden activa",
Activo = true });
        context.SaveChanges();

        var cancelada = new OrdenFabricacion
        {
            NumeroOrden = "OF-M2-CANCELADA",
            Cliente = "Cliente Histórico",
            FechaCreacion = DateTime.UtcNow,
            Estado = "Cancelada"
        };
        context.OrdenesFabricacion.Add(cancelada);
        context.SaveChanges();
        var visibles = context.OrdenesFabricacion.Count();
        var todas = context.OrdenesFabricacion.IgnoreQueryFilters().Count();
        global::System.Console.WriteLine($"2.10 OK | Visibles: {visibles} | Sin filtro: {todas}");
    }
}

```

Explicación línea a línea del código acumulativo 2.10

Las referencias usan la numeración real de `M02/PROYECTO/2.10/Program.cs`. Se explican todas las líneas no vacías del archivo que compila.

Línea 1: `using System.ComponentModel.DataAnnotations;` → importa el espacio de nombres necesario para este archivo.

Línea 2: `using System.ComponentModel.DataAnnotations.Schema;` → importa el espacio de nombres necesario para este archivo.

Línea 3: `using Microsoft.EntityFrameworkCore;` → importa el espacio de nombres necesario para este archivo.

Línea 4: `using Microsoft.Extensions.Configuration;` → importa el espacio de nombres necesario para este archivo.

Línea 5: `using Microsoft.Extensions.DependencyInjection;` → importa el espacio de nombres necesario para este archivo.

Línea 6: `using Microsoft.Extensions.Logging;` → importa el espacio de nombres necesario para este archivo.

Línea 8: `namespace AceriaData.ConsoleApp;` → declara el espacio de nombres del estado acumulativo.

Línea 10: `[Table("OrdenesFabricacion")]` → fija mediante Data Annotations la tabla relacional de la entidad siguiente.

Línea 11: `public class OrdenFabricacion` → declara la clase `OrdenFabricacion` que forma parte del estado acumulativo.

Línea 12: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 13: `[Key]` → marca la propiedad siguiente como clave primaria simple.

Línea 14: `public int Id { get; set; }` → declara la propiedad `Id` de tipo `int` usada por el modelo o la lógica de negocio.

Línea 15: `[Required]` → marca la propiedad siguiente como requerida.

Línea 16: `[MaxLength(50)]` → limita la longitud máxima persistida de la propiedad siguiente.

Línea 17: `public string NumeroOrden { get; set; } = string.Empty;` → declara la propiedad `NumeroOrden` de tipo `string` usada por el modelo o la lógica de negocio.

Línea 18: `[Required]` → marca la propiedad siguiente como requerida.

Línea 19: `[MaxLength(200)]` → limita la longitud máxima persistida de la propiedad siguiente.

Línea 20: `public string Cliente { get; set; } = string.Empty;` → declara la propiedad Cliente de tipo string usada por el modelo o la lógica de negocio.

Línea 21: `public DateTime FechaCreacion { get; set; }` → declara la propiedad FechaCreacion de tipo DateTime usada por el modelo o la lógica de negocio.

Línea 22: `public DateTime? FechaEntrega { get; set; }` → declara la propiedad FechaEntrega de tipo DateTime? usada por el modelo o la lógica de negocio.

Línea 23: `[Required]` → marca la propiedad siguiente como requerida.

Línea 24: `[MaxLength(50)]` → limita la longitud máxima persistida de la propiedad siguiente.

Línea 25: `public string Estado { get; set; } = "Pendiente";` → declara la propiedad Estado de tipo string usada por el modelo o la lógica de negocio.

Línea 26: `[MaxLength(500)]` → limita la longitud máxima persistida de la propiedad siguiente.

Línea 27: `public string? Observaciones { get; set; }` → declara la propiedad Observaciones de tipo string? usada por el modelo o la lógica de negocio.

Línea 28: `public List<PlanchaAcero> Planchas { get; set; } = new();` → declara la propiedad Planchas de tipo List usada por el modelo o la lógica de negocio.

Línea 29: `public DetalleOrden? Detalle { get; set; }` → declara la propiedad Detalle de tipo DetalleOrden? usada por el modelo o la lógica de negocio.

Línea 30: `public CertificadoCalidad? Certificado { get; set; }` → declara la propiedad Certificado de tipo CertificadoCalidad? usada por el modelo o la lógica de negocio.

Línea 31: `public List<OrdenAleacion> OrdenesAleaciones { get; set; } = new();` → declara la propiedad OrdenesAleaciones de tipo List usada por el modelo o la lógica de negocio.

Línea 32: `}` → cierra el bloque de código actual.

Línea 34: `[Table("PlanchasAcero")]` → fija mediante Data Annotations la tabla relacional de la entidad siguiente.

Línea 35: `public class PlanchaAcero` → declara la clase PlanchaAcero que forma parte del estado acumulativo.

Línea 36: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 37: `[Key]` → marca la propiedad siguiente como clave primaria simple.

Línea 38: `public int Id { get; set; }` → declara la propiedad Id de tipo int usada por el modelo o la lógica de negocio.

Línea 39: `public int OrdenId { get; set; }` → declara la propiedad OrdenId de tipo int usada por el modelo o la lógica de negocio.

Línea 40: `public double Espesor { get; set; }` → declara la propiedad Espesor de tipo double usada por el modelo o la lógica de negocio.

Línea 41: `public double Ancho { get; set; }` → declara la propiedad Ancho de tipo double usada por el modelo o la lógica de negocio.

Línea 42: `public double Largo { get; set; }` → declara la propiedad Largo de tipo double usada por el modelo o la lógica de negocio.

Línea 43: `[Precision(18, 3)]` → configura precisión y escala para la propiedad numérica siguiente.

Línea 44: `public decimal Peso { get; set; }` → declara la propiedad Peso de tipo decimal usada por el modelo o la lógica de negocio.

Línea 45: `public bool Activa { get; set; } = true;` → declara la propiedad Activa de tipo bool usada por el modelo o la lógica de negocio.

Línea 46: `[ForeignKey(nameof(OrdenId))]` → vincula la navegación siguiente con su clave foránea.

Línea 47: `public OrdenFabricacion Orden { get; set; } = null!;` → declara la propiedad Orden de tipo OrdenFabricacion usada por el modelo o la lógica de negocio.

Línea 48: `}` → cierra el bloque de código actual.

Línea 50: `[Table("Aleaciones")]` → fija mediante Data Annotations la tabla relacional de la entidad siguiente.

Línea 51: `public class Aleacion` → declara la clase Aleacion que forma parte del estado acumulativo.

Línea 52: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 53: `[Key]` → marca la propiedad siguiente como clave primaria simple.

Línea 54: `public int Id { get; set; }` → declara la propiedad Id de tipo int usada por el modelo o la lógica de negocio.

Línea 55: `[Required]` → marca la propiedad siguiente como requerida.

Línea 56: `[MaxLength(100)]` → limita la longitud máxima persistida de la propiedad siguiente.

Línea 57: `public string Nombre { get; set; } = string.Empty;` → declara la propiedad Nombre de tipo string usada por el modelo o la lógica de negocio.

Línea 58: `[Required]` → marca la propiedad siguiente como requerida.

Línea 59: `[MaxLength(20)]` → limita la longitud máxima persistida de la propiedad siguiente.

Línea 60: `public string Codigo { get; set; } = string.Empty;` → declara la propiedad Codigo de tipo string usada por el modelo o la lógica de negocio.

Línea 61: `public double PorcentajeCarbono { get; set; }` → declara la propiedad PorcentajeCarbono de tipo double usada por el modelo o la lógica de negocio.

Línea 62: `public double PorcentajeManganeso { get; set; }` → declara la propiedad PorcentajeManganeso de tipo double usada por el modelo o la lógica de negocio.

Línea 63: `[MaxLength(500)]` → limita la longitud máxima persistida de la propiedad siguiente.

Línea 64: `public string? Descripcion { get; set; }` → declara la propiedad Descripcion de tipo string? usada por el modelo o la lógica de negocio.

Línea 65: `public List<OrdenAleacion> OrdenesAleaciones { get; set; } = new();` → declara la propiedad OrdenesAleaciones de tipo List usada por el modelo o la lógica de negocio.

Línea 66: `}` → cierra el bloque de código actual.

Línea 68: `[Table("EstadosOrden")]` → fija mediante Data Annotations la tabla relacional de la entidad siguiente.

Línea 69: `public class EstadoOrden` → declara la clase EstadoOrden que forma parte del estado acumulativo.

Línea 70: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 71: `[Key]` → marca la propiedad siguiente como clave primaria simple.

Línea 72: `public int Id { get; set; }` → declara la propiedad Id de tipo int usada por el modelo o la lógica de negocio.

Línea 73: `[Required]` → marca la propiedad siguiente como requerida.

Línea 74: `[MaxLength(50)]` → limita la longitud máxima persistida de la propiedad siguiente.

Línea 75: `public string Nombre { get; set; } = string.Empty;` → declara la propiedad Nombre de tipo string usada por el modelo o la lógica de negocio.

Línea 76: `[MaxLength(250)]` → limita la longitud máxima persistida de la propiedad siguiente.

Línea 77: `public string Descripcion { get; set; } = string.Empty;` → declara la propiedad Descripcion de tipo string usada por el modelo o la lógica de negocio.

Línea 78: `public bool Activo { get; set; } = true;` → declara la propiedad Activo de tipo bool usada por el modelo o la lógica de negocio.

Línea 79: `}` → cierra el bloque de código actual.

Línea 81: `[Table("DetallesOrden")]` → fija mediante Data Annotations la tabla relacional de la entidad siguiente.

Línea 82: `public class DetalleOrden` → declara la clase DetalleOrden que forma parte del estado acumulativo.

Línea 83: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 84: `[Key]` → marca la propiedad siguiente como clave primaria simple.

Línea 85: `public int Id { get; set; }` → declara la propiedad Id de tipo int usada por el modelo o la lógica de negocio.

Línea 86: `public int OrdenId { get; set; }` → declara la propiedad OrdenId de tipo int usada por el modelo o la lógica de negocio.

Línea 87: `[Required]` → marca la propiedad siguiente como requerida.

Línea 88: `[MaxLength(200)]` → limita la longitud máxima persistida de la propiedad siguiente.

Línea 89: `public string ComposicionQuimica { get; set; } = string.Empty;` → declara la propiedad ComposicionQuimica de tipo string usada por el modelo o la lógica de negocio.

Línea 90: `public double TemperaturaColada { get; set; }` → declara la propiedad TemperaturaColada de tipo double usada por el modelo o la lógica de negocio.

Línea 91: `[MaxLength(500)]` → limita la longitud máxima persistida de la propiedad siguiente.

Línea 92: `public string? Notas { get; set; }` → declara la propiedad Notas de tipo string? usada por el modelo o la lógica de negocio.

Línea 93: `[ForeignKey(nameof(OrdenId))]` → vincula la navegación siguiente con su clave foránea.

Línea 94: `public OrdenFabricacion Orden { get; set; } = null!;` → declara la propiedad Orden de tipo OrdenFabricacion usada por el modelo o la lógica de negocio.

Línea 95: `}` → cierra el bloque de código actual.

Línea 97: `[Table("CertificadosCalidad")]` → fija mediante Data Annotations la tabla relacional de la entidad siguiente.

Línea 98: `public class CertificadoCalidad` → declara la clase CertificadoCalidad que forma parte del estado acumulativo.

Línea 99: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 100: `[Key]` → marca la propiedad siguiente como clave primaria simple.

Línea 101: `public int Id { get; set; }` → declara la propiedad Id de tipo int usada por el modelo o la lógica de negocio.

Línea 102: `public int OrdenId { get; set; }` → declara la propiedad OrdenId de tipo int usada por el modelo o la lógica de negocio.

Línea 103: `[Required]` → marca la propiedad siguiente como requerida.

Línea 104: `[MaxLength(50)]` → limita la longitud máxima persistida de la propiedad siguiente.

Línea 105: `public string NumeroCertificado { get; set; } = string.Empty;` → declara la propiedad `NumeroCertificado` de tipo `string` usada por el modelo o la lógica de negocio.

Línea 106: `public DateTime FechaEmision { get; set; }` → declara la propiedad `FechaEmision` de tipo `DateTime` usada por el modelo o la lógica de negocio.

Línea 107: `[Required]` → marca la propiedad siguiente como requerida.

Línea 108: `[MaxLength(100)]` → limita la longitud máxima persistida de la propiedad siguiente.

Línea 109: `public string OrganismoCertificador { get; set; } = string.Empty;` → declara la propiedad `OrganismoCertificador` de tipo `string` usada por el modelo o la lógica de negocio.

Línea 110: `[ForeignKey(nameof(OrdenId))]` → vincula la navegación siguiente con su clave foránea.

Línea 111: `public OrdenFabricacion Orden { get; set; } = null!;` → declara la propiedad `Orden` de tipo `OrdenFabricacion` usada por el modelo o la lógica de negocio.

Línea 112: `}` → cierra el bloque de código actual.

Línea 114: `[PrimaryKey(nameof(OrdenFabricacionId), nameof(AleacionId))]` → declara una clave primaria compuesta.

Línea 115: `[Table("OrdenesAleaciones")]` → fija mediante Data Annotations la tabla relacional de la entidad siguiente.

Línea 116: `public class OrdenAleacion` → declara la clase `OrdenAleacion` que forma parte del estado acumulativo.

Línea 117: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 118: `public int OrdenFabricacionId { get; set; }` → declara la propiedad `OrdenFabricacionId` de tipo `int` usada por el modelo o la lógica de negocio.

Línea 119: `public int AleacionId { get; set; }` → declara la propiedad `AleacionId` de tipo `int` usada por el modelo o la lógica de negocio.

Línea 120: `public DateTime FechaAsignacion { get; set; } = DateTime.Now;` → declara la propiedad `FechaAsignacion` de tipo `DateTime` usada por el modelo o la lógica de negocio.

Línea 121: `[Precision(18, 3)]` → configura precisión y escala para la propiedad numérica siguiente.

Línea 122: `public decimal CantidadUtilizada { get; set; }` → declara la propiedad `CantidadUtilizada` de tipo `decimal` usada por el modelo o la lógica de negocio.

Línea 123: `[MaxLength(20)]` → limita la longitud máxima persistida de la propiedad siguiente.

Línea 124: `public string EstadoRelacion { get; set; } = "Activa";` → declara la propiedad `EstadoRelacion` de tipo `string` usada por el modelo o la lógica de negocio.

Línea 125: `public OrdenFabricacion Orden { get; set; } = null!;` → declara la propiedad `Orden` de tipo `OrdenFabricacion` usada por el modelo o la lógica de negocio.

Línea 126: `public Aleacion Aleacion { get; set; } = null!;` → declara la propiedad `Aleacion` de tipo `Aleacion` usada por el modelo o la lógica de negocio.

Línea 127: `}` → cierra el bloque de código actual.

Línea 129: `public class AceriaDbContext : DbContext` → declara la clase `AceriaDbContext` que forma parte del estado acumulativo.

Línea 130: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 131:

`public AceriaDbContext(DbContextOptions<AceriaDbContext> options) : base(options) { }` → recibe las opciones del `DbContext` y las pasa a la clase base.

Línea 133: `public DbSet<OrdenFabricacion> OrdenesFabricacion => Set<OrdenFabricacion>();` → expone un conjunto de entidades para consultas y persistencia.

Línea 134: `public DbSet<PlanchaAcero> PlanchasAcero => Set<PlanchaAcero>();` → expone un conjunto de entidades para consultas y persistencia.

Línea 135: `public DbSet<Aleacion> Aleaciones => Set<Aleacion>();` → expone un conjunto de entidades para consultas y persistencia.

Línea 136: `public DbSet<EstadoOrden> EstadosOrden => Set<EstadoOrden>();` → expone un conjunto de entidades para consultas y persistencia.

Línea 137: `public DbSet<DetalleOrden> DetallesOrden => Set<DetalleOrden>();` → expone un conjunto de entidades para consultas y persistencia.

Línea 138: `public DbSet<CertificadoCalidad> CertificadosCalidad => Set<CertificadoCalidad>();` → expone un conjunto de entidades para consultas y persistencia.

Línea 139: `public DbSet<OrdenAleacion> OrdenesAleaciones => Set<OrdenAleacion>();` → expone un conjunto de entidades para consultas y persistencia.

Línea 142: `protected override void OnModelCreating(ModelBuilder modelBuilder)` → entra en la configuración explícita del modelo.

Línea 143: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 144: `base.OnModelCreating(modelBuilder);` → entra en la configuración explícita del modelo.

Línea 145: `modelBuilder.Entity<OrdenFabricacion>(entity =>` → selecciona una entidad para configurarla con Fluent API.

Línea 146: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 147: `entity.ToTable("OrdenesFabricacion");` → asigna explícitamente la tabla usada por la entidad.

Línea 148: `entity.HasKey(o => o.Id);` → define la clave primaria.

Línea 149: `entity.Property(o => o.NumeroOrden).IsRequired().HasMaxLength(50);` → selecciona una propiedad escalar para continuar su configuración.

Línea 150: `entity.Property(o => o.Cliente).IsRequired().HasMaxLength(200);` → selecciona una propiedad escalar para continuar su configuración.

Línea 151: `entity.Property(o => o.FechaCreacion).HasDefaultValueSql("GETDATE()");` → selecciona una propiedad escalar para continuar su configuración.

Línea 152: `entity.Property(o => o.Estado).IsRequired().HasMaxLength(50).HasDefaultValue("Pendiente");` → selecciona una propiedad escalar para continuar su configuración.

Línea 153: `entity.Property(o => o.Observaciones).HasMaxLength(500);` → selecciona una propiedad escalar para continuar su configuración.

Línea 154: `entity.HasAlternateKey(o => o.NumeroOrden).HasName("AK_OrdenesFabricacion_NumeroOrden");` → define una clave alternativa única.

Línea 155: `entity.HasIndex(o => o.Cliente).HasDatabaseName("IX_OrdenesFabricacion_Cliente");` → define un índice sobre las propiedades indicadas.

Línea 156: `entity.HasIndex(o => new { o.Cliente, o.FechaCreacion }).HasDatabaseName("IX_OrdenesFabricacion_Cliente_FechaCreacion");` → define un índice sobre las propiedades indicadas.

Línea 157: `entity.HasIndex(o => o.FechaEntrega).HasFilter("[Estado] = 'Pendiente']").HasDatabaseName("IX_OrdenesFabricacion_FechaEntrega_Pendientes");` → define un índice sobre las propiedades indicadas.

Línea 158: `entity.HasIndex(o => o.Estado).IncludeProperties(o => new { o.NumeroOrden, o.Cliente, o.FechaCreacion }).HasDatabaseName("IX_OrdenesFabricacion_Estado_Incluye");` → define un índice sobre las propiedades indicadas.

Línea 159: `entity.HasQueryFilter(o => o.Estado != "Cancelada");` → aplica un filtro global de consulta.

Línea 160: `});` → cierra la lambda o configuración encadenada y finaliza la llamada.

Línea 161: `modelBuilder.Entity<PlanchaAcero>(entity =>` → selecciona una entidad para configurarla con Fluent API.

Línea 162: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 163: `entity.ToTable("PlanchasAcero", t =>` → asigna explícitamente la tabla usada por la entidad.

Línea 164: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 165: `t.HasCheckConstraint("CK_PlanchasAcero_Espesor", "[Espesor] > 0");` → añade una restricción CHECK en SQL Server.

Línea 166: `t.HasCheckConstraint("CK_PlanchasAcero_Ancho", "[Ancho] > 0");` → añade una restricción CHECK en SQL Server.

Línea 167: `t.HasCheckConstraint("CK_PlanchasAcero_Largo", "[Largo] > 0");` → añade una restricción CHECK en SQL Server.

Línea 168: `t.HasCheckConstraint("CK_PlanchasAcero_Peso", "[Peso] > 0");` → añade una restricción CHECK en SQL Server.

Línea 169: `});` → cierra la lambda o configuración encadenada y finaliza la llamada.

Línea 170: `entity.HasKey(x => x.Id);` → define la clave primaria.

Línea 171: `entity.Property(x => x.Peso).HasPrecision(18, 3);` → selecciona una propiedad escalar para continuar su configuración.

Línea 172: `entity.Property(x => x.Activa).HasDefaultValue(true);` → selecciona una propiedad escalar para continuar su configuración.

Línea 173: `entity.HasOne(x => x.Orden)` → inicia la configuración de una navegación de referencia.

Línea 174: `.WithMany(o => o.Planchas)` → establece el extremo de colección de una relación uno-a-muchos.

Línea 175: `.HasForeignKey(x => x.OrdenId)` → declara explícitamente la clave foránea.

Línea 176: `.OnDelete(DeleteBehavior.Cascade)` → define el comportamiento de eliminación de la relación.

Línea 177: `.IsRequired();` → marca la propiedad o relación como obligatoria.

Línea 178: `entity.HasIndex(x => new { x.OrdenId, x.Activa }).HasDatabaseName("IX_PlanchasAcero_OrdenId_Activa");` → define un índice sobre las propiedades indicadas.

Línea 179: `entity.HasIndex(x => x.Espesor).HasFilter("[Activa] = 1").HasDatabaseName("IX_PlanchasAcero_Espesor_Activas");` → define un índice sobre las propiedades indicadas.

Línea 180: `entity.HasQueryFilter(x => x.Activa);` → aplica un filtro global de consulta.

Línea 181: `});` → cierra la lambda o configuración encadenada y finaliza la llamada.

Línea 182: `modelBuilder.Entity<Aleacion>(entity =>` → selecciona una entidad para configurarla con Fluent API.

Línea 183: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 184: `entity.ToTable("Aleaciones", t =>` → asigna explícitamente la tabla usada por la entidad.

Línea 185: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 186: `t.HasCheckConstraint("CK_Aleaciones_PorcentajeCarbono", "[PorcentajeCarbono] >= 0 AND [PorcentajeCarbono] <= 2");` → añade una restricción CHECK en SQL Server.

Línea 187: `t.HasCheckConstraint("CK_Aleaciones_PorcentajeManganeso", "[PorcentajeManganeso] >= 0 AND [PorcentajeManganeso] <= 5");` → añade una restricción CHECK en SQL Server.

Línea 188: `});` → cierra la lambda o configuración encadenada y finaliza la llamada.

Línea 189: `entity.HasKey(a => a.Id);` → define la clave primaria.

Línea 190: `entity.Property(a => a.Nombre).IsRequired().HasMaxLength(100);` → selecciona una propiedad escalar para continuar su configuración.

Línea 191: `entity.Property(a => a.Codigo).IsRequired().HasMaxLength(20);` → selecciona una propiedad escalar para continuar su configuración.

Línea 192: `entity.Property(a => a.Descripcion).HasMaxLength(500);` → selecciona una propiedad escalar para continuar su configuración.

Línea 193: `entity.HasAlternateKey(a => a.Codigo).HasName("AK_Aleaciones_Codigo");` → define una clave alternativa única.

Línea 194: `entity.HasIndex(a => a.Nombre).HasDatabaseName("IX_Aleaciones_Nombre");` → define un índice sobre las propiedades indicadas.

Línea 195: `entity.HasIndex(a => new { a.PorcentajeCarbono, a.PorcentajeManganeso }).HasDatabaseName("IX_Aleaciones_Porcentajes");` → define un índice sobre las propiedades indicadas.

Línea 196: `});` → cierra la lambda o configuración encadenada y finaliza la llamada.

Línea 197: `modelBuilder.Entity<EstadoOrden>(entity =>` → selecciona una entidad para configurarla con Fluent API.

Línea 198: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 199: `entity.ToTable("EstadosOrden");` → asigna explícitamente la tabla usada por la entidad.

Línea 200: `entity.HasKey(e => e.Id);` → define la clave primaria.

Línea 201: `entity.Property(e => e.Nombre).IsRequired().HasMaxLength(50);` → selecciona una propiedad escalar para continuar su configuración.

Línea 202: `entity.Property(e => e.Descripcion).HasMaxLength(250);` → selecciona una propiedad escalar para continuar su configuración.

Línea 203: `entity.Property(e => e.Activo).HasDefaultValue(true);` → selecciona una propiedad escalar para continuar su configuración.

Línea 204: `entity.HasQueryFilter(e => e.Activo);` → aplica un filtro global de consulta.

Línea 205: `});` → cierra la lambda o configuración encadenada y finaliza la llamada.

Línea 206: `modelBuilder.Entity<DetalleOrden>(entity =>` → selecciona una entidad para configurarla con Fluent API.

Línea 207: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 208: `entity.ToTable("DetallesOrden");` → asigna explícitamente la tabla usada por la entidad.

Línea 209: `entity.HasKey(d => d.Id);` → define la clave primaria.

Línea 210: `entity.Property(d => d.ComposicionQuimica).IsRequired().HasMaxLength(200);` → selecciona una propiedad escalar para continuar su configuración.

Línea 211: `entity.Property(d => d.Notas).HasMaxLength(500);` → selecciona una propiedad escalar para continuar su configuración.

Línea 212: `entity.HasOne(d => d.Orden)` → inicia la configuración de una navegación de referencia.

Línea 213: `.WithOne(o => o.Detalle)` → establece el extremo de referencia de una relación uno-a-uno.

Línea 214: `.HasForeignKey<DetalleOrden>(d => d.OrdenId)` → declara explícitamente la clave foránea.

Línea 215: `.OnDelete(DeleteBehavior.Cascade)` → define el comportamiento de eliminación de la relación.

Línea 216: `.IsRequired();` → marca la propiedad o relación como obligatoria.

Línea 217: `});` → cierra la lambda o configuración encadenada y finaliza la llamada.

Línea 219: `modelBuilder.Entity<CertificadoCalidad>(entity =>` → selecciona una entidad para configurarla con Fluent API.

Línea 220: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 221: `entity.ToTable("CertificadosCalidad");` → asigna explícitamente la tabla usada por la entidad.

Línea 222: `entity.HasKey(c => c.Id);` → define la clave primaria.

Línea 223: `entity.Property(c => c.NumeroCertificado).IsRequired().HasMaxLength(50);` → selecciona una propiedad escalar para continuar su configuración.

Línea 224: `entity.Property(c => c.OrganismoCertificador).IsRequired().HasMaxLength(100);` → selecciona una propiedad escalar para continuar su configuración.

Línea 225: `entity.HasAlternateKey(c => c.NumeroCertificado).HasName("AK_CertificadosCalidad_NumeroCertificado");` → define una clave alternativa única.

Línea 226: `entity.HasIndex(c => c.FechaEmision).HasDatabaseName("IX_CertificadosCalidad_FechaEmision");` → define un índice sobre las propiedades indicadas.

Línea 227: `entity.HasOne(c => c.Orden)` → inicia la configuración de una navegación de referencia.

Línea 228: `.WithOne(o => o.Certificado)` → establece el extremo de referencia de una relación uno-a-uno.

Línea 229: `.HasForeignKey<CertificadoCalidad>(c => c.OrdenId)` → declara explícitamente la clave foránea.

Línea 230: `.OnDelete(DeleteBehavior.Cascade)` → define el comportamiento de eliminación de la relación.

Línea 231: `.IsRequired();` → marca la propiedad o relación como obligatoria.

Línea 232: `});` → cierra la lambda o configuración encadenada y finaliza la llamada.

Línea 233: `modelBuilder.Entity<OrdenAleacion>(entity =>` → selecciona una entidad para configurarla con Fluent API.

Línea 234: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 235: `entity.ToTable("OrdenesAleaciones");` → asigna explícitamente la tabla usada por la entidad.

Línea 236: `entity.HasKey(x => new { x.OrdenFabricacionId, x.AleacionId });` → define la clave primaria.

Línea 237: `entity.Property(x => x.FechaAsignacion).HasDefaultValueSql("GETDATE()");` → selecciona una propiedad escalar para continuar su configuración.

Línea 238: `entity.Property(x => x.CantidadUtilizada).HasPrecision(18, 3);` → selecciona una propiedad escalar para continuar su configuración.

Línea 239: `entity.Property(x => x.EstadoRelacion).IsRequired().HasMaxLength(20).HasDefaultValue("Activa");` → selecciona una propiedad escalar para continuar su configuración.

Línea 240: `entity.HasOne(x => x.Orden)` → inicia la configuración de una navegación de referencia.

Línea 241: `.WithMany(o => o.OrdenesAleaciones)` → establece el extremo de colección de una relación uno-a-muchos.

Línea 242: `.HasForeignKey(x => x.OrdenFabricacionId)` → declara explícitamente la clave foránea.

Línea 243: `.OnDelete(DeleteBehavior.Cascade);` → define el comportamiento de eliminación de la relación.

Línea 244: `entity.HasOne(x => x.Aleacion)` → inicia la configuración de una navegación de referencia.

Línea 245: `.WithMany(a => a.OrdenesAleaciones)` → establece el extremo de colección de una relación uno-a-muchos.

Línea 246: `.HasForeignKey(x => x.AleacionId)` → declara explícitamente la clave foránea.

Línea 247: `.OnDelete(DeleteBehavior.Restrict);` → define el comportamiento de eliminación de la relación.

Línea 248: `entity.HasIndex(x => x.AleacionId).HasDatabaseName("IX_OrdenesAleaciones_AleacionId");` → define un índice sobre las propiedades indicadas.

Línea 249: `entity.HasIndex(x => x.EstadoRelacion).HasFilter("[EstadoRelacion] = 'Activa'").HasDatabaseName("IX_OrdenesAleaciones_EstadoRelacion_Activas");` → define un índice sobre las propiedades indicadas.

Línea 250: `entity.HasQueryFilter(x => x.EstadoRelacion == "Activa");` → aplica un filtro global de consulta.

Línea 251: `});` → cierra la lambda o configuración encadenada y finaliza la llamada.

Línea 252: `}` → cierra el bloque de código actual.

Línea 253: `}` → cierra el bloque de código actual.

Línea 255: `public interface IOrdenRepositorio` → declara el contrato `IOrdenRepositorio` usado por la aplicación.

Línea 256: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 257: `List<OrdenFabricacion> ObtenerTodas();` → declara en el contrato la operación que debe devolver todas las órdenes visibles para el repositorio.

Línea 258: `OrdenFabricacion? ObtenerPorId(int id);` → declara en el contrato la búsqueda de una orden por su clave primaria y permite devolver null si no existe.

Línea 259: `OrdenFabricacion? ObtenerPorNumero(string numeroOrden);` → declara en el contrato la búsqueda por el identificador de negocio `NumeroOrden`.

Línea 260: `void Agregar(OrdenFabricacion orden);` → declara la operación que incorpora una orden al repositorio para que EF Core la siga como nueva.

Línea 261: `void Eliminar(OrdenFabricacion orden);` → declara la operación que marca una orden para eliminación física cuando el escenario lo requiere.

Línea 262: `int Guardar();` → declara la operación que confirma los cambios pendientes y devuelve el número de entradas afectadas.

Línea 263: `}` → cierra el bloque de código actual.

Línea 265: `public sealed class OrdenRepositorio : IOrdenRepositorio` → declara la clase `OrdenRepositorio` que forma parte del estado acumulativo.

Línea 266: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 267: `private readonly AceriaDbContext _context;` → declara el campo privado `_context` de tipo `AceriaDbContext`.

Línea 268: `public OrdenRepositorio(AceriaDbContext context) => _context = context;` → inyecta `AceriaDbContext` en el repositorio y lo conserva como dependencia de persistencia.

Línea 270: `public List<OrdenFabricacion> ObtenerTodas() => _context.OrdenesFabricacion.OrderBy(o => o.Id).ToList();` → ordena el resultado antes de materializarlo.

Línea 271: `public OrdenFabricacion? ObtenerPorId(int id) => _context.OrdenesFabricacion.Find(id);` → implementa la búsqueda por clave primaria mediante `Find`, aprovechando primero el `ChangeTracker` y después la base de datos.

Línea 272: `public OrdenFabricacion? ObtenerPorNumero(string numeroOrden) => _context.OrdenesFabricacion.FirstOrDefault(o => o.NumeroOrden == numeroOrden);` → recupera la primera coincidencia o `null`.

Línea 273: `public void Agregar(OrdenFabricacion orden) => _context.OrdenesFabricacion.Add(orden);` → registra una entidad nueva para insertarla al guardar.

Línea 274: `public void Eliminar(OrdenFabricacion orden) => _context.OrdenesFabricacion.Remove(orden);` → marca una entidad para borrado al guardar.

Línea 275: `public int Guardar() => _context.SaveChanges();` → confirma en SQL Server los cambios del `ChangeTracker`.

Línea 276: `}` → cierra el bloque de código actual.

Línea 278: `public interface IServicioOrdenes` → declara el contrato `IServicioOrdenes` usado por la aplicación.

Línea 279: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 280: `string ObtenerResumen(int ordenId);` → declara el servicio que construye un resumen de una orden a partir de su identificador.

Línea 281: `}` → cierra el bloque de código actual.

Línea 283: `public sealed class ServicioOrdenes : IServicioOrdenes` → declara la clase `ServicioOrdenes` que forma parte del estado acumulativo.

Línea 284: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 285: `private readonly IOrdenRepositorio _repositorio;` → declara el campo privado `_repositorio` de tipo `IOrdenRepositorio`.

Línea 286: `private readonly AceriaDbContext _context;` → declara el campo privado `_context` de tipo `AceriaDbContext`.

Línea 287: `public ServicioOrdenes(IOrdenRepositorio repositorio, AceriaDbContext context)` → declara el constructor que recibe por inyección el repositorio de órdenes y el `DbContext`.

Línea 288: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 289: `_repositorio = repositorio;` → guarda en el servicio el repositorio inyectado para reutilizarlo en sus operaciones.

Línea 290: `_context = context;` → guarda el DbContext inyectado para consultar los datos necesarios durante el servicio.

Línea 291: `}` → cierra el bloque de código actual.

Línea 293: `public string ObtenerResumen(int ordenId)` → declara el método que obtiene la orden y compone el resumen solicitado por el servicio.

Línea 294: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 295: `var orden = _repositorio.ObtenerPorId(ordenId);` → declara una variable local y almacena el resultado de la expresión.

Línea 296: `if (orden is null) return $"Orden {ordenId} no encontrada";` → evalúa una condición antes de ejecutar el bloque asociado.

Línea 297: `var totalPlanchas = _context.PlanchasAcero.Count(x => x.OrdenId == ordenId);` → cuenta las filas que cumplen la consulta.

Línea 298: `return $"Orden {orden.NumeroOrden} | Cliente: {orden.Cliente} | Planchas: {totalPlanchas}";` → devuelve el resultado al código llamador.

Línea 299: `}` → cierra el bloque de código actual.

Línea 300: `}` → cierra el bloque de código actual.

Línea 302: `public static class Program` → declara la clase Program que forma parte del estado acumulativo.

Línea 303: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 304: `public static void Main()` → declara el punto de entrada ejecutable de la aplicación de consola.

Línea 305: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 306: `var configuration = new ConfigurationBuilder()` → inicia la construcción de la configuración de la aplicación.

Línea 307: `.SetBasePath(AppContext.BaseDirectory)` → establece la ruta base para los archivos de configuración.

Línea 308: `.AddJsonFile("appsettings.json", optional: false, reloadOnChange: false)` → incorpora appsettings.json.

Línea 309: `.AddEnvironmentVariables()` → incorpora variables de entorno.

Línea 310: `.Build();` → materializa el objeto construido por la cadena anterior.

Línea 312: `var connectionString = configuration.GetConnectionString("AceriaDB")` → obtiene la cadena AceriaDB desde configuración.

Línea 313: `?? throw new InvalidOperationException("No se encontró la cadena de conexión 'AceriaDB'.");` → interrumpe el arranque con un error explícito si la configuración no contiene ConnectionStrings:AceriaDB.

Línea 315: `var services = new ServiceCollection();` → crea la colección de servicios.

Línea 316: `services.AddDbContext<AceriaDbContext>(options =>` → registra el DbContext en inyección de dependencias.

Línea 317: `options` → continúa la configuración del DbContext sobre el parámetro options recibido por la lambda de AddDbContext.

Línea 318: `.UseSqlServer(connectionString, sql =>` → selecciona SQL Server como proveedor de EF Core.

Línea 319: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 320: `sql.EnableRetryOnFailure(5, TimeSpan.FromSeconds(10), null);` → habilita reintentos ante fallos transitorios.

Línea 321: `sql.CommandTimeout(60);` → establece el timeout de los comandos SQL.

Línea 322: `})` → cierra la lambda de configuración actual y devuelve el control a la llamada encadenada que la contiene.

Línea 323: `.LogTo(global::System.Console.WriteLine, new[]
{ "Microsoft.EntityFrameworkCore.Database.Command" }, LogLevel.Information)` → configura el logging de EF Core.

Línea 324: `.EnableDetailedErrors();` → activa errores detallados.

Línea 326: `services.AddScoped<IOrdenRepositorio, OrdenRepositorio>();` → registra un servicio con ciclo de vida scoped.

Línea 327: `services.AddScoped<IServicioOrdenes, ServicioOrdenes>();` → registra un servicio con ciclo de vida scoped.

Línea 329: `using var provider = services.BuildServiceProvider(new ServiceProviderOptions` → construye el ServiceProvider con validación de servicios y garantiza su liberación al salir del ámbito mediante using var.

Línea 330: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 331: `ValidateScopes = true,` → activa la validación de ámbitos para detectar dependencias scoped usadas desde ámbitos incorrectos.

Línea 332: `ValidateOnBuild = true` → obliga al contenedor a validar las resoluciones registradas al construir el ServiceProvider.

Línea 333: `});` → cierra la lambda o configuración encadenada y finaliza la llamada.

Línea 335: `using var scope = provider.CreateScope();` → crea un ámbito de DI para resolver servicios scoped y lo libera automáticamente al terminar el bloque.

Línea 336: `var context = scope.ServiceProvider.GetRequiredService<AceriaDbContext>();` → resuelve un servicio obligatorio del contenedor.

Línea 337: `context.Database.EnsureDeleted();` → reinicia la base del laboratorio para una ejecución reproducible.

Línea 338: `context.Database.Migrate();` → aplica el historial de migraciones pendiente.

Línea 340: `var orden = new OrdenFabricacion` → declara una variable local y almacena el resultado de la expresión.

Línea 341: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 342: `NumeroOrden = "OF-M2-0001",` → asigna el identificador de negocio usado por la orden de validación acumulativa del módulo.

Línea 343: `Cliente = "Constructora del Norte",` → asigna el cliente de la orden utilizada en el escenario práctico del punto.

Línea 344: `FechaCreacion = DateTime.UtcNow,` → asigna a la orden la fecha de creación en UTC para el escenario reproducible del punto.

Línea 345: `FechaEntrega = DateTime.Today.AddDays(14),` → establece una fecha de entrega prevista catorce días después de la fecha local actual.

Línea 346: `Estado = "Pendiente"`, → inicializa la orden con el estado Pendiente, coherente con el flujo del laboratorio.

Línea 347: `Observaciones = "Orden de validación M2"`, → añade texto opcional para comprobar el mapeo de Observaciones.

Línea 348: `Planchas =` → inicia el inicializador de la colección de planchas asociadas a la orden.

Línea 349: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 350: `new PlanchaAcero` → crea una nueva plancha dentro del inicializador de la colección Planchas de la orden.

Línea 351: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 352: `Espesor = 10.5`, → asigna a la plancha un espesor positivo para satisfacer las reglas del modelo.

Línea 353: `Ancho = 1500`, → asigna el ancho de la plancha usada en el escenario de persistencia.

Línea 354: `Largo = 3000`, → asigna el largo de la plancha usada en el escenario de persistencia.

Línea 355: `Peso = 371.250m`, → asigna un peso decimal con tres posiciones para validar la precisión configurada.

Línea 356: `Activa = true`, → marca la plancha como activa para que sea visible en los estados que aplican ese criterio.

Línea 357: `}` → cierra el bloque de código actual.

Línea 358: `},` → continúa el inicializador o la llamada del bloque actual.

Línea 359: `Detalle = new DetalleOrden` → crea el detalle uno-a-uno que se persistirá junto con la orden principal.

Línea 360: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 361: `ComposicionQuimica = "C: 0.45%, Mn: 0.75%"`, → asigna la composición química utilizada para verificar la entidad DetalleOrden.

Línea 362: `TemperaturaColada = 1550.5`, → asigna la temperatura de colada del detalle de fabricación.

Línea 363: `Notas = "Colada principal"` → asigna las notas opcionales del detalle asociado a la orden.

Línea 364: `},` → continúa el inicializador o la llamada del bloque actual.

Línea 365: `Certificado = new CertificadoCalidad` → crea el certificado uno-a-uno asociado a la orden del escenario.

Línea 366: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 367: `NumeroCertificado = "CERT-0001"`, → asigna el identificador natural del certificado de calidad.

Línea 368: `FechaEmision = DateTime.Today`, → asigna la fecha de emisión del certificado usado en la prueba.

Línea 369: `OrganismoCertificador = "Laboratorio Aceria"` → asigna el organismo que emite el certificado de calidad.

Línea 370: `},` → continúa el inicializador o la llamada del bloque actual.

Línea 371: `};` → cierra el inicializador del objeto y finaliza la instrucción.

Línea 373: `var aleacion = new Aleacion` → declara una variable local y almacena el resultado de la expresión.

Línea 374: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 375: `Nombre = "AISI 1045"`, → asigna el nombre comercial de la aleación utilizada en el escenario.

Línea 376: `Codigo = "A1045"`, → asigna el código natural único de la aleación.

Línea 377: `PorcentajeCarbono = 0.45`, → asigna el porcentaje de carbono dentro del rango permitido por la restricción del modelo.

Línea 378: `PorcentajeManganeso = 0.75`, → asigna el porcentaje de manganeso dentro del rango permitido por la restricción del modelo.

Línea 379: `Descripcion = "Acero medio en carbono"` → asigna una descripción opcional a la aleación.

Línea 380: `};` → cierra el inicializador del objeto y finaliza la instrucción.

Línea 381: `orden.OrdenesAleaciones.Add(new OrdenAleacion` → registra una entidad nueva para insertarla al guardar.

Línea 382: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 383: `Aleacion = aleacion`, → conecta la entidad intermedia `OrdenAleacion` con la aleación creada para el escenario.

Línea 384: `CantidadUtilizada = 1500.500m`, → registra la cantidad utilizada con tres decimales, coherente con `HasPrecision(18, 3)`.

Línea 385: `EstadoRelacion = "Activa"` → marca como activa la relación entre la orden y la aleación.

Línea 386: `});` → cierra la lambda o configuración encadenada y finaliza la llamada.

Línea 387: `context.OrdenesFabricacion.Add(orden);` → registra una entidad nueva para insertarla al guardar.

Línea 388: `context.EstadosOrden.Add(new EstadoOrden { Nombre = "Pendiente", Descripcion = "Orden activa", Activo = true });` → registra una entidad nueva para insertarla al guardar.

Línea 389: `context.SaveChanges();` → confirma en SQL Server los cambios del `ChangeTracker`.

Línea 391: `var cancelada = new OrdenFabricacion` → declara una variable local y almacena el resultado de la expresión.

Línea 392: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 393: `NumeroOrden = "0F-M2-CANCELADA"`, → asigna el número de la orden de control que se crea con estado Cancelada.

Línea 394: `Cliente = "Cliente Histórico"`, → asigna el cliente de la orden histórica utilizada para comprobar el filtro global.

Línea 395: `FechaCreacion = DateTime.UtcNow`, → asigna a la orden la fecha de creación en UTC para el escenario reproducible del punto.

Línea 396: `Estado = "Cancelada"` → establece el estado Cancelada para verificar que `HasQueryFilter` excluye la orden de las consultas normales.

Línea 397: `};` → cierra el inicializador del objeto y finaliza la instrucción.

Línea 398: `context.OrdenesFabricacion.Add(cancelada);` → registra una entidad nueva para insertarla al guardar.

Línea 399: `context.SaveChanges();` → confirma en SQL Server los cambios del `ChangeTracker`.

Línea 400: `var visibles = context.OrdenesFabricacion.Count();` → cuenta las filas que cumplen la consulta.

Línea 401: `var todas = context.OrdenesFabricacion.IgnoreQueryFilters().Count();` → omite de forma explícita los filtros globales para esta consulta.

Línea 402: `global::System.Console.WriteLine($"2.10 OK | Visibles: {visibles} | Sin filtro: {todas}");` → escribe la evidencia de ejecución usada para validar este estado.

Línea 403: `}` → cierra el bloque de código actual.

Línea 404: `}` → cierra el bloque de código actual.